CSE 213 — Software Engineering

Interaction Diagrams — Sequence and Collaboration

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — sequence / collaboration questions
Lesson 16 — Sequence Diagram Notation — Lec 8 sl. 1–10; message types, frames
Lesson 17 — From a Use Case to a Sequence Diagram — Lec 9 sl. 1–5; withdraw money, MRI, chapters.com
Lesson 18 — Collaboration (Communication) Diagrams — Lec 9 sl. 6–8, Lec 10 sl. 1–7; grocery invoice, check-out books
Quick Notes — Interaction Diagrams — revision + key diagrams

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lesson 16: 'Lec 8 - sequence diagram.pdf' (10 slides). Lessons 17–18: 'Lec 9 - seq-collaboration.pdf' (8 slides) and 'Lec 10 - additional collaboration.pdf' (7 scanned slides from Bennett et al. ch. 9). Sequence diagrams are written as text: participants in a header row, then numbered messages 'A -> B : msg()' (synchronous), 'A ->> B' (asynchronous), 'B --> A' (return).

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2016-17 (CSE 307)	Q2(c) Sequence diagram for 'Withdraw money' (13) [L17]
2014-15 (CSE 307)	Q7(c) Two types of interaction diagrams with example (9⅔) [L18]
2013-14 (CSE 307)	Q7(b) MRI centre referral — sequence diagram (16⅔) [L17]
2012-13 (CSE 307)	Q5(c) Word processor / print spooler / printer — collaboration diagram + activity diagram of print file (9) [L18] · Q6(a) Grocery invoice collaboration diagram → sequence diagram, stereotypes, Invoice class (16) [L18]
2011-12 (CSE 307)	Q2(c) Types of messages in a sequence diagram (12) [L16]
2010-11 (CSE 307)	Q1(a) Check Out Books use case → collaboration and class diagrams (24⅔) [L18] · Q2(a) chapters.com book purchase — sequence diagram (12) [L17]

EXAM FOCUS Older-paper topic (no question since 2016-17), but a reliable 12–25 marks when asked: draw a sequence diagram from a narrative (2016-17, 2013-14, 2010-11), convert collaboration ↔ sequence (2012-13), or list message types (2011-12). Learn the notation table of Lesson 16 and the boundary–control–entity layout used by Lec 9.

Lesson 16 — Sequence Diagram Notation

Slides 1–10 (Lec 8 - sequence diagram)

A sequence diagram is an interaction diagram that models **one scenario** of a use case: which messages are sent, between which objects, and in what order (sl. 1). Horizontal axis = participants; vertical axis = time, downward (sl. 2). The scenario usually starts with an unattached 'found message' from the actor.

Element	Notation	Slide
Object / participant	box with underlined name:Class (name optional: :Client)	sl. 4
Lifeline	dashed vertical line under the box — the object's life during the scenario	sl. 4
Synchronous message	solid line, filled arrowhead; caller waits	sl. 5
Asynchronous message	solid line, open (stick) arrowhead; caller continues	sl. 5
Return message	dashed line, open arrowhead	sl. 5
Activation	thin rectangle on the lifeline while the method is on the stack; nest it for self-calls	sl. 6
Creation	arrow labelled new / «create» to a box drawn lower down	sl. 7
Deletion	X at the end of the lifeline	sl. 7
Frames	opt [cond] (if), alt [cond] / [else] separated by a dashed line (if/else), loop [cond]	sl. 8

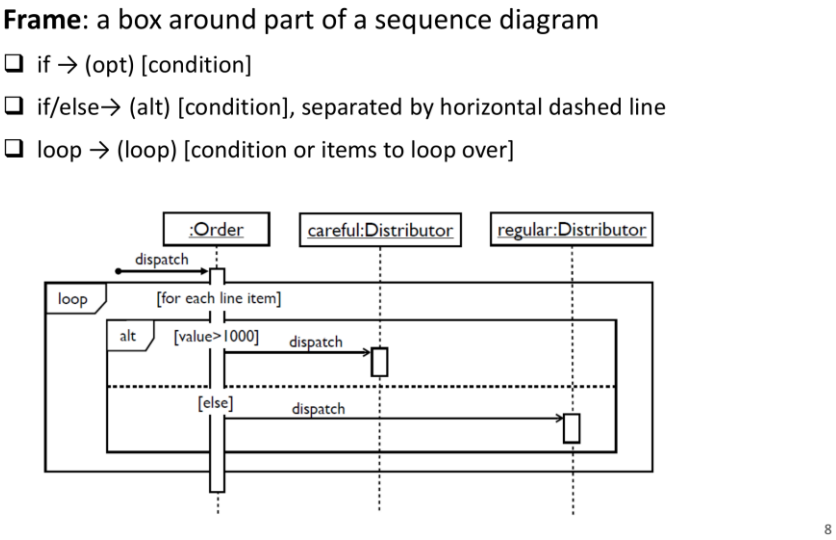


Figure 16.1 — Frames: loop over line items with alt [value > 1000] (sl. 8)

Worked example — check email (sl. 3)

```
Actor  :Client                      :Server
 |-- checkEmail -->|                          (found message)
 |                 |-- sendUnsentEmail -------->|
 |                 |<-- response (ack) ---------|
 |                 |-- newEmail --------------->|
 |                 |<-- response ---------------|
 |                 |-- [newEmail] getEmail ---->|          (guarded message)
 |                 |-- deleteOldEmail --------->|
```

Practice — Lesson 16

(a) Actual — 2011-12 Q2(c).

(a) What are the different types of messages in a sequence diagram? Discuss them briefly with examples. (12)

✓ Solution

(a) Six message kinds, each with arrow and example:

Message	Arrow	Example
Synchronous (call)	solid line, filled head; sender blocks until return	:ATM → :Bank : verifyPIN(pin)
Asynchronous (signal)	solid line, open head; sender continues	:WordProcessor →> :PrintSpooler : print(file)
Return / reply	dashed line, open head	:Bank - -> :ATM : ok
Create	dashed/solid arrow «create» / new to the new object's box	:Handler → :Query : new
Destroy	message ending in X on the target lifeline	:Session → :Cart : «destroy»
Self (reflexive)	arrow back to the same lifeline, nested activation	:Order → :Order : computeTotal()
Found / lost	found: from a filled dot (unknown sender); lost: to a dot (never arrives)	found: checkEmail from the user

Lesson 17 — From a Use Case to a Sequence Diagram

Slides 1–5 (Lec 9 - seq-collaboration)

1. The Register use case (sl. 1–3)

Typical course of events (17 steps): student opens the registration interface, the system asks for id and password, shows unfinished courses, the student selects courses, the system checks prerequisites (deselecting failed ones), checks total credit hours, notifies the adviser, who views details and authorises; the system updates the registration and prints the course card.

The sequence diagrams use **boundary** objects (:MainMenuUI, :RegisterUI), a **control** object (:Register) and **entity** objects (:Student, :Course, :RegInfo). Diagram 1 (sl. 2): itemSelected() → [selectedItem = register] Register() creates the control, RegisterUI(), showLogin(), login(id, pw) → verifyLogin(id, pw) → Student() / getStudentInfo(id); [pwAccepted = false] showLogin(). Diagram 2 (sl. 3): [pwAccepted = true] *getCourseList(),* getRegInfo(id), prepareUnfinishedCourseList(), showCourseList(*Courses); loop selectCourse(Cid) → checkPreRequisite() → [PRcompleted = true] / addToList(RegInfo) or showError(type); showConfirmation(type).

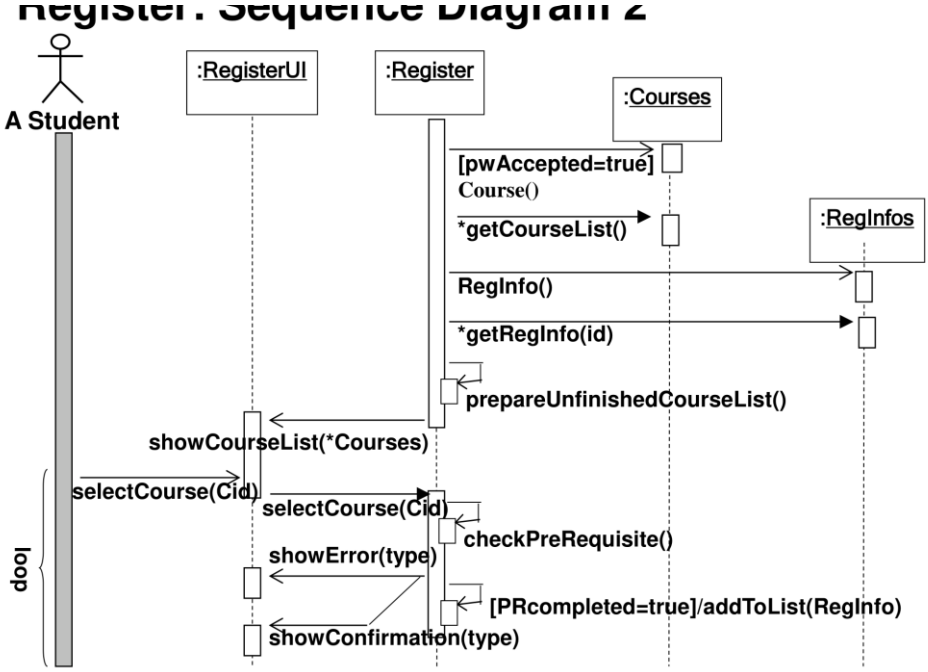


Figure 17.1 — Register sequence diagram 2 with a loop frame (sl. 3)

sl. 4 (printer): CustomerP → PrinterP : PrintFile; PrinterP → Printer : GetStatus; branching [Ready] Print, [Busy] PutInQueue, [OutOfService] CallRepair; asynchronous Ready(file) / GetNext from the queue. sl. 5: 'Add a new advert to a campaign' with loop frames and creation of newAd:Advert.

Exam Approach

1. Number the narrative into steps; each verb between two parties becomes one message.
2. Choose participants left to right: actor, boundary (UI), control, entities, external systems.
3. Use alt for if/else (e.g. limit exceeded), opt for optional steps, loop for repetition.
4. Show activations, returns of important values and creations (new).

Practice — Lesson 17

(a) Actual — 2016-17 Q2(c). (b) Actual — 2013-14 Q7(b). (c) Actual — 2010-11 Q2(a). (d) Actual — 2014-15 Q8(b) (sequence part).

(a) Bank: withdrawal ≤ 1,00,000 per transaction, ≤ 3,00,000 and ≤ 5 withdrawals per day. Draw the sequence diagram for 'Withdraw money'. (13)

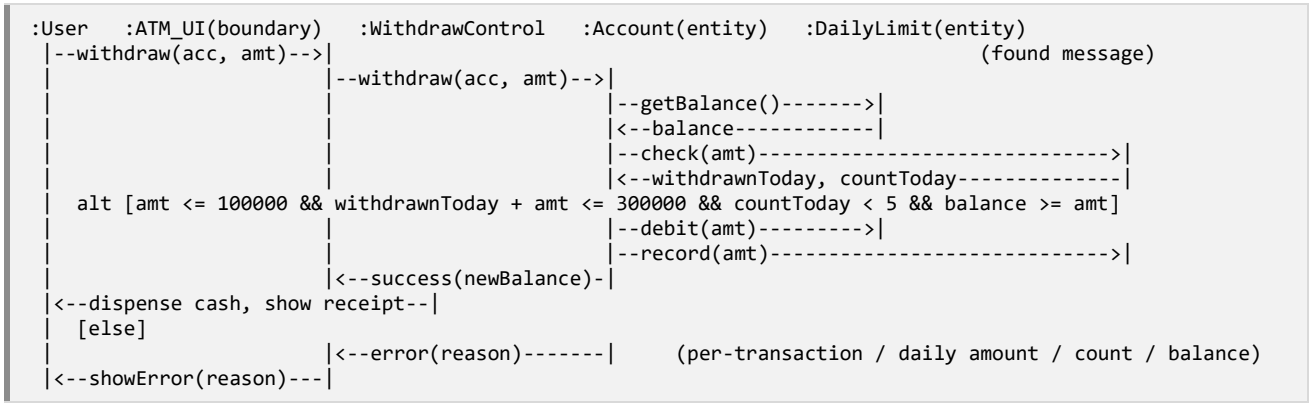
(b) MRI centre: a patient brings a doctor's referral; an intake clerk reviews it for completeness, checks the Active Doctor file, signs the form, prepares a 3-part MRI assignment (machine and technician); copy 3 mailed to the doctor, copy 2 filed by date with the referral, copy 1 to the Imaging Department; a technician performs the procedure and updates the form. Sequence diagram. (16⅔)

(c) chapters.com: customer selects a book, gives card information; the system gets bank authorisation and, if positive, confirms; the order goes to the orders department; when the book is available it is shipped; the order department charges the card by informing the bank. Show activation intervals. (12)

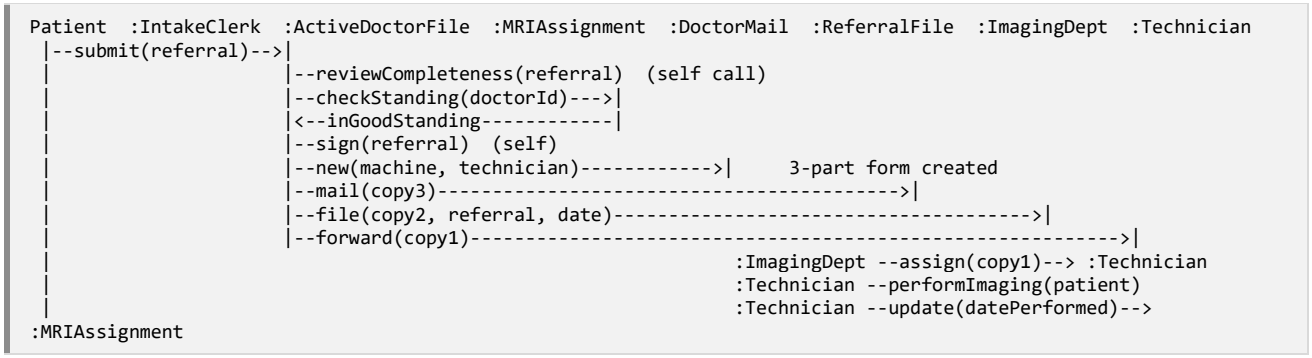
(d) Music-CD lending: sequence diagram for lending a CD to a friend. (10 of 22)

✓ Solution

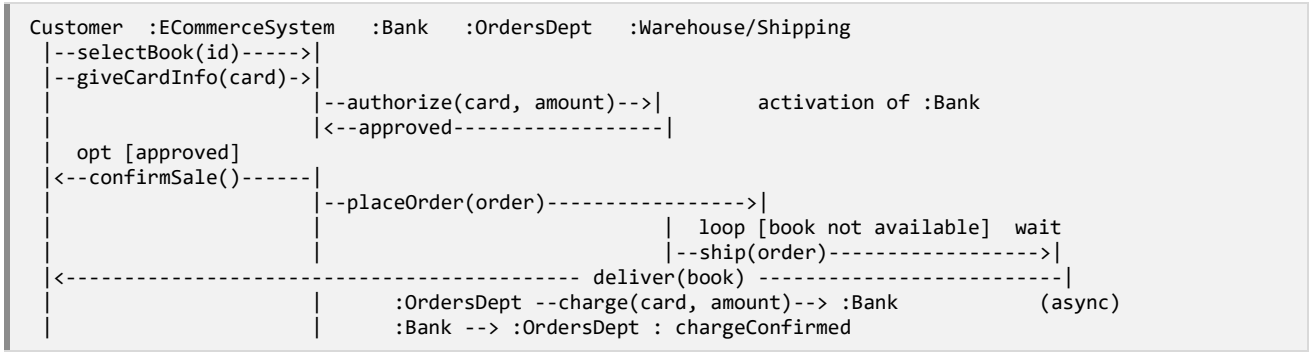
(a)



(b)



(c) Activation bars run from the moment a participant receives a message until it replies.



(d)

```
Owner  :LendingUI  :CDCopy  :Friend  :Loan
 |--lend(copyNo, friendId)-->|
 |              |--isAvailable()-->| <--yes--|
 |              |--find(friendId)--------------->|
 |              |--new Loan(copy, friend, today, feePerDay)------------->|
 |              |--markLent()----->|
 |<--confirmation(loanId)--|
```

Lesson 18 — Collaboration (Communication) Diagrams

Lec 9 sl. 6–8; Lec 10 sl. 1–7

1. What they are (Lec 9 sl. 6)

- Hold the **same information** as sequence diagrams, but show the **links** between the objects.
- There is no time axis; order is given by sequence numbers on the messages.
- Numbers are nested (3, 3.1, 3.1.1) to show which message is sent while handling which.

Register collaboration diagram (sl. 7): RegisterUI, AuthorizeRegistrationUI (boundaries), Register (control), Student, RegistrationInfo, Result, Course (entities); messages 2 showLogin(), 3 login(), 4 verifyLogin(), 7 showCourseList(), 8 selectCourse(), 9 verifyPreReq(), 10 getResult(), 11 showPRError(), 12–13 finalize(), 14a/14b showRegInfo()/showTCError(), 15–16 authorize(), 18 updateRegInfo()/showConfirmation(). Concurrent alternatives use letters (14a, 14b).

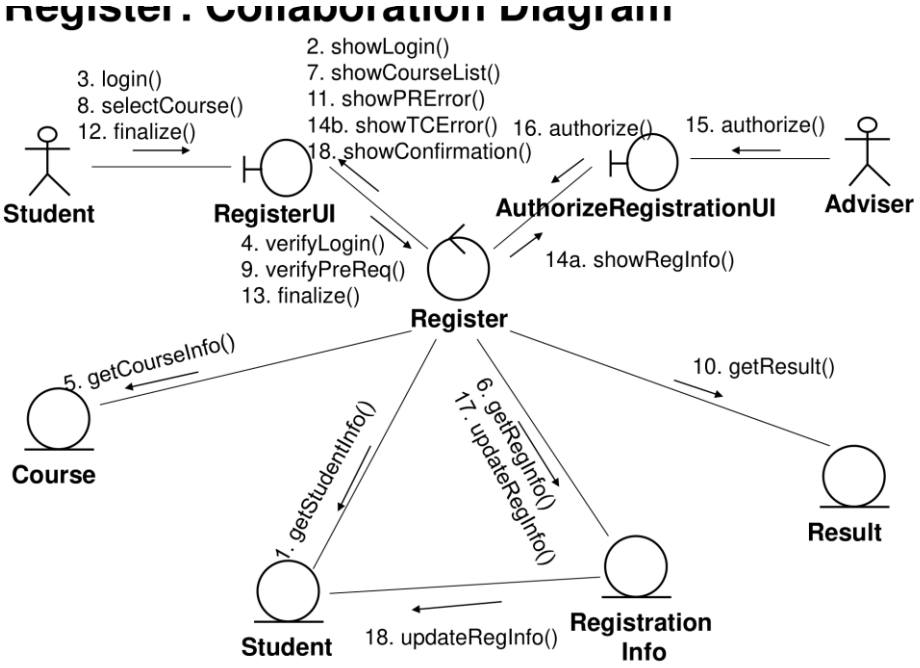


Figure 18.1 — Register collaboration diagram (sl. 7)

2. Model consistency (sl. 8)

Every operation in a message must exist in the receiver's class with the same signature, and the sender must hold a reference to the receiver — through an association, or because another object passed it.

3. Additional examples (Lec 10, scanned)

- Add a new campaign: 1 *getClient(), 2 startInterface(), 3 selectClient(), 4 showClientCampaigns(), 5 getClientCampaigns(), 6–7 createNewCampaign(), 8 createCampaign(), 9 addNewCampaign().
- Assign staff contact: *getClient(),* getStaffDetails(), selectClient(), assignStaffContact() forwarded to Client.
- Check campaign budget: nested numbering 3.1, 3.1.1, 3.1.2, 4.1, 4.1.1, 4.1.1.1 *getCost(), 4.1.1.2 getOverheads().
- Record completion: 4.1.2 printCompletionForm() on :CompleteCampaignPI; Add a new advert: 5.1.1.1 creates the Advert. Slide 7 gives the equivalent sequence diagram.

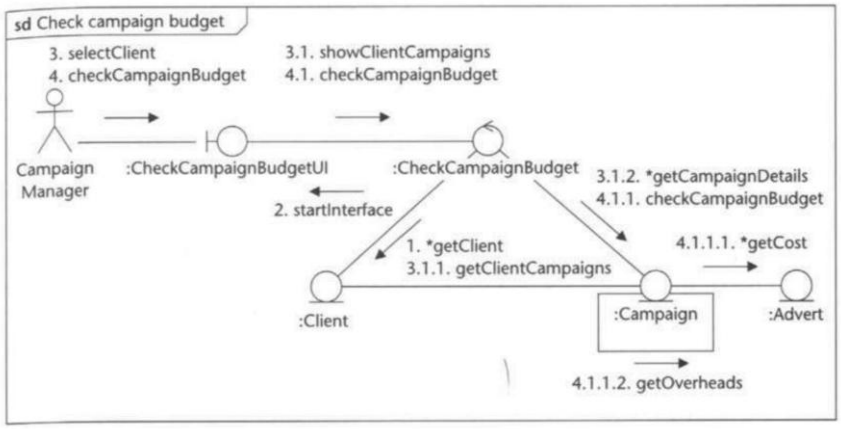


Figure 18.2 — Check campaign budget: nested sequence numbers (Lec 10 sl. 4)

Practice — Lesson 18

(a) Actual — 2014-15 Q7(c). (b) Actual — 2012-13 Q6(a). (c) Actual — 2012-13 Q5(c). (d) Actual — 2010-11 Q1(a).

(a) Explain two different types of interaction diagrams with example. (9⅔)

(b) Grocery sales system collaboration diagram for creating an invoice: 1: InvoiceNo := NewInvoice(); 2: SearchProduct(query); 2.1: searchProduct(query); 3: SelectProduct(pcode, qty); 3.1: GetPrice(pcode); 3.2: [product available] CalcAmount(qty, price); 4: SaveInvoice(); 4.1: SaveInvoice(); 4.1.1: UpdateQuantity(pcode, qty) among :Cashier, :CreateInvoice, :Invoice, :Product. (i) Draw the sequence diagram (ii) add stereotypes (iii) give the UML class Invoice with everything the diagram implies. (10+2+4)

(c) A word processor (active object) creates a print file and asynchronously asks a print spooler (active object) to print it; the spooler repeatedly reads a block and sends it to the printer (active object) by procedure call. Draw the collaboration diagram and an activity diagram of the print-file operation. (9)

(d) Check Out Books use case (worker, patron, books, due date by faculty/student, non-circulating books): draw a relatively complete collaboration diagram and class diagram with associations. (24⅔)

✓ Solution

(a) Sequence diagram — time-ordered, lifelines and activations; best for showing the order and timing of messages (e.g. check email, sl. 3 of Lec 8). **Collaboration (communication) diagram** — objects and links in space, messages numbered 1, 1.1, 2 …; best for showing which objects are connected (e.g. Register, Lec 9 sl. 7). Both are semantically equivalent; tools can convert one into the other.

(b)(i) From the numbers: 2.1 is sent by :CreateInvoice to :Product while handling 2, etc.

```
:Cashier            :CreateInvoice «control»       :Invoice «entity»        :Product «entity»
 |--1: NewInvoice()-------->|
 |<--InvoiceNo--------------|
 |--2: SearchProduct(query)->|--2.1: searchProduct(query)------------------------>|
 |--3: SelectProduct(pcode,qty)->|--3.1: GetPrice(pcode)---------------------------->|
 |                          |<--price-------------------------------------------|
 |                          |--3.2: [product available] CalcAmount(qty, price)-->|  (on :Invoice)
 |--4: SaveInvoice()------->|--4.1: SaveInvoice()------>|
 |                          |                           |--4.1.1: UpdateQuantity(pcode, qty)-->|
```

(ii) :Cashier is the actor (or «boundary» if it is the cashier's screen); :CreateInvoice «control»; :Invoice «entity»; :Product «entity».

(iii) Only what the diagram shows: Invoice receives CalcAmount(qty, price) and SaveInvoice(), and sends UpdateQuantity to Product (so Invoice is associated with Product); NewInvoice() returns an InvoiceNo.

```
+-----------------------------------------+
|              Invoice «entity»           |
+-----------------------------------------+
| invoiceNo                               |
| amount (computed by CalcAmount)         |
+-----------------------------------------+
| + CalcAmount(qty, price)                |
| + SaveInvoice()                         |
+-----------------------------------------+
Invoice * ---- * Product   (Invoice sends UpdateQuantity(pcode, qty) to Product)
```

(c) Active objects are drawn with a thick border; the asynchronous message has a half (stick) arrowhead.

```
COLLABORATION DIAGRAM
 [:WordProcessor] (active) ---1: «create»---> [:PrintFile]
 [:WordProcessor] ---2: print(file)  (asynchronous)---> [:PrintSpooler] (active)
 [:PrintSpooler]  ---2.1 *[until end of file]: readBlock()---> [:PrintFile]
 [:PrintSpooler]  ---2.2 *[for each block]: printBlock(block)  (procedure call)---> [:Printer] (active)

ACTIVITY DIAGRAM – print file operation
 ● -> (Create print file) -> (Request spooler to print) -> (Read next block) -> <block read?>
      [yes] -> (Send block to printer) -> back to (Read next block)
      [end of file] -> (Close print file) -> ◉
```

(d) Collaboration diagram (one checkout, steps 1–5 repeated per book):

```
Worker --1: identifyPatron(patronId)--> [:CheckOutUI] --1.1: checkOutStart(patronId)-->
[:CheckOutController]
[:CheckOutController] --1.2: p := findPatron(patronId)--> [:PatronCatalog] ; --1.3: allowed := canBorrow()-
-> [p:Patron]
   1.4a [not allowed]: showProhibition(reason) --> [:CheckOutUI]
*[for each book]  2: enterBook(bookId) --> [:CheckOutController]
   2.1: c := findCopy(bookId) --> [:BookCatalog] ;  2.2: circ := canCirculate() --> [c:Copy]
   2.3a [not circ]: showNonCirculating() --> [:CheckOutUI]
   2.3b [circ]: due := calcDueDate(p.type) (self) ; 2.4: new Loan(p, c, due) --> [l:Loan]
   2.5: markCheckedOut() --> [c:Copy]  ;  2.6: showDueDate(due) --> [:CheckOutUI]
3: done() --> [:CheckOutController]
```

```
Class diagram
 Patron {abstract} [id, name, finesDue; canBorrow()]   <|-- FacultyPatron (loan period: academic year)
                                                        <|-- StudentPatron (loan period: limited days)
 Patron 1 ---- 0..* Loan [checkoutDate, dueDate]  ---- 1 Copy [barcode, circulating: boolean;
canCirculate()]
 Book [isbn, title] 1 ◆---- 1..* Copy        BookCatalog 1 ◇---- * Book      PatronCatalog 1 ◇---- *
Patron
 Worker [id, name] 1 ---- * Loan (records ▶)
 CheckOutController ---> PatronCatalog, BookCatalog, Loan     CheckOutUI ---> CheckOutController
```

Quick Notes — Interaction Diagrams

Revision points for Lessons 16–18.

L16 · Sequence notation

- Box :Class (underlined), dashed lifeline, activation bar, X = destroyed, new = created.
- Sync = filled head; async = stick head; return = dashed.
- Frames: opt, alt (dashed separator), loop.

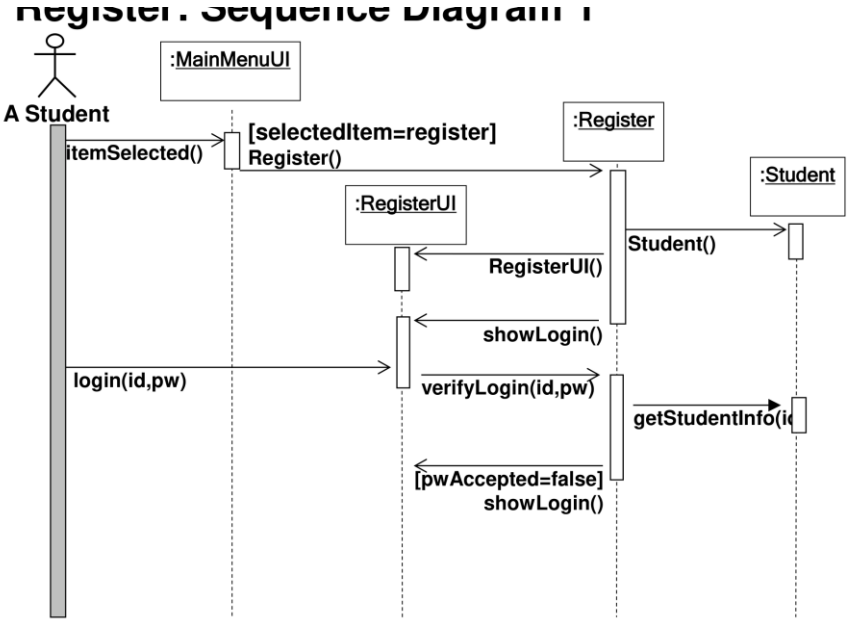
L17 · From use case

- Actor → boundary → control → entity; one message per step of the narrative.
- Register example: verifyLogin, getCourseList, loop selectCourse / checkPreRequisite.

L18 · Collaboration

- Same information, no time axis, links shown, nested numbers 3.1.1; letters for alternatives (14a/14b).
- Consistency: operation exists in receiver's class; sender has a link.
- Grocery invoice: Invoice has CalcAmount(qty, price), SaveInvoice(); associated with Product.

Key Diagrams to Reproduce



Register sequence diagram 1 (Lec 9 sl. 2)